Changelog — v2.1 (2026-03-21)

> Complete architectural overhaul of the Hyperledger Fabric Guarantee of Origin platform. Transforms the prototype into a production-ready full-stack application with tiered organizations, OO chaincode, and a React UI.

Summary

Metric	v1 (Original)	v2.1 (Current)
Chaincode files	1 monolithic file (2,350 lines)	20 files across 4 packages
Smart contracts	1 unnamed contract	6 named contracts
Bug fixes	—	12 bugs fixed
Network orgs	4 hardcoded (buyer, eproducer, hproducer, issuer)	Tiered roles (N issuers, N producers, N consumers)
Frontend	None	React 18 + Express.js REST API
On-chain assets	GOs + Certificates	GOs + Certificates + Devices + Counters + Roles
Counter persistence	In-memory (lost on restart)	On-chain state
Access control	Inconsistent ABAC	Unified RBAC + collection-level ACLs

Chaincode (Go)

New Package Structure

The monolithic `conversion.go` (2,350 lines) has been decomposed into an object-oriented structure:

```
chaincode/
├── main.go                 — Registers 6 named contracts
├── assets/                 — 5 files: type definitions
│   ├── electricity_go.go   — ElectricityGO, ElectricityGOPrivateDetails
│   ├── hydrogen_go.go      — GreenHydrogenGO, GreenHydrogenGOPrivateDetails, Backlog
│   ├── certificate.go      — CancellationStatement{E,H}, ConsumptionDeclaration{E,H}
│   ├── counter.go          — On-chain persistent counter with GetNextID()
│   └── device.go           — Device struct, status/type constants
├── access/                 — 3 files: access control
│   ├── roles.go            — GetOrgRole, RequireRole, RequireAnyRole, IsIssuer, IsProducer
│   ├── abac.go             — GetAttribute, AssertAttribute, GetClientMSPID
│   └── collections.go      — GetOwnCollection, GetCollectionForOrg, ValidateCollectionAccess
├── util/                   — 3 files: shared helpers
│   ├── iterator.go         — ConstructEGOsFromIterator, ConstructHGOsFromIterator
│   ├── validate.go         — UnmarshalTransient, ValidatePositive, ValidateNonEmpty, GetTimestamp
│   └── split.go            — SplitElectricityGO, SplitHydrogenGO, Write/Delete ledger helpers
├── contracts/              — 6 files: smart contract logic
│   ├── issuance.go         — CreateElectricityGO, CreateHydrogenGO
│   ├── transfer.go         — TransferEGO, TransferEGOByAmount, TransferHGOByAmount
│   ├── conversion.go       — AddHydrogenToBacklog, IssuehGO, QueryHydrogenBacklog
│   ├── cancellation.go     — ClaimRenewableAttributesElectricity/Hydrogen, VerifyCancellationStatement
│   ├── query.go            — GetCurrentEGOsList/HGOsList, ReadPublic/Private, QueryByAmount
│   └── device_mgmt.go      — RegisterDevice, RevokeDevice, SuspendDevice, ReactivateDevice, InitLedger
└── go.mod                  — Module: github.com/JonaAlbrecht/HLF-GOconversionissuance-JA-MA/chaincode
```

6 Named Contracts

Clients invoke functions with a namespace prefix (e.g., `issuance:CreateElectricityGO`):

Contract	Functions	Domain
`issuance`	`CreateElectricityGO`, `CreateHydrogenGO`	GO minting from device readings

Contract	Functions	Domain
`transfer`	`TransferEGO`, `TransferEGOByAmount`, `TransferHGOByAmount`	Inter-org GO transfers with splitting
`conversion`	`AddHydrogenToBacklog`, `IssuehGO`, `QueryHydrogenBacklog`	Electricity → hydrogen conversion
`cancellation`	`ClaimRenewableAttributesElectricity`, `ClaimRenewableAttributesHydrogen`, `VerifyCancellationStatement`, `SetGOEndorsementPolicy`	GO retirement & certificate creation
`query`	`GetCurrentEGOsList`, `GetCurrentHGOsList`, `ReadPublicEGO/HGO`, `ReadPrivateEGO/HGO`, `ReadCancellationStatement*`, `ReadConsumptionDeclaration*`, `QueryPrivateEGOsByAmountMWh`, `QueryPrivateHGOsByAmount`	Read-only queries
`device`	`RegisterDevice`, `GetDevice`, `ListDevices`, `RevokeDevice`, `SuspendDevice`, `ReactivateDevice`, `RegisterOrgRole`, `InitLedger`	Device lifecycle + network bootstrap

New Feature: On-Chain Device Management

Devices (SmartMeters, OutputMeters) are now first-class on-chain assets, replacing the previous X.509 ABAC-only approach:

- **Register**: Issuer registers devices with type, owner org, energy carriers, and attribute map
- **Lifecycle**: Devices can be suspended and reactivated without re-enrollment
- **Revoke**: Permanently disable compromised devices
- **Validation**: Issuance contracts check device status and attributes before minting GOs

New Feature: Role Registry

Organizations register their tier/role on-chain via `InitLedger` or `RegisterOrgRole`:

```
orgRole_issuer1MSP  → "issuer"
orgRole_producer1MSP → "producer"
orgRole_consumer1MSP → "consumer"
```

All access control flows through `access.RequireRole()` / `access.RequireAnyRole()`.

🐛 Bug Fixes (12)

#	Bug Description	Root Cause	Fix	File(s)
1	**ID collisions after restart**	In-memory `Count` variables reset to 0 on chaincode container restart	On-chain persistent counters via `GetNextID()`	`assets/counter.go`
2	**Race condition in counter**	`mu.Unlock()` then immediate `GetState()` in Go goroutine model	Fabric's deterministic execution + single `PutState` per transaction	`assets/counter.go`
3	**IssuehGO overwrites accumulated emissions**	Final hydrogen emissions assigned from last eGO instead of running total	Track `accumulatedHEmissions` as a running sum	`contracts/conversion.go`

#	Bug Description	Root Cause	Fix	File(s)
4	**IssuehGO skips final eGO**	Missing `else` branch when `eGO.AmountMWh >= backlog.UsedMWh`	Added else clause to handle exact-match and over-match cases	`contracts/conversion.go`
5	**QueryPrivateeGOs `remove()` skips elements**	Removing from a slice while iterating forward shifts indices	Replaced with simple iteration (no in-place removal in query)	`contracts/query.go`
6	**No bounds checking → panics**	Functions accepted negative amounts, empty strings without validation	Centralized `ValidatePositive()` and `ValidateNonEmpty()` at entry	`util/validate.go`, all contracts
7	**`organization` attr vs `GetMSPID()` inconsistency**	Some functions used custom "organization" X.509 attribute, others used MSPID	Unified to always use `GetClientMSPID()` for collection derivation	`access/abac.go`
8	**Remainder eGO loses CreationDateTime**	Split operation created a new GO with `GetTimestamp()` instead of preserving original	Preserve original `CreationDateTime` on remainder after split	`util/split.go`, `contracts/cancellation.go`
9	**ConsumptionDeclarations shared across split hGOs**	Slice reference shared between original and remainder after hydrogen split	Deep-copy declarations with `make()` + `copy()` before assignment	`util/split.go`
10	**ConsumptionDeclarationHydrogen.DateTime is string**	Mixed types: electricity declarations used `int64`, hydrogen used `string`	Unified all `DateTime` fields to `int64` (Unix timestamp)	`assets/certificate.go`
11	**VerifyCancellationStatement hash never matches**	Compared sequential counter-based `AssetID` against SHA-256 hash of data	Removed broken hash comparison; verify by direct ledger lookup	`contracts/cancellation.go`
12	**No access control on private reads**	Any org could read any other org's private collection data	Added `ValidateCollectionAccess()` — issuers can audit all, others restricted to own	`contracts/query.go`, `access/collections.go`

🌐 Network Configuration

Tiered Architecture (replaces hardcoded orgs)

Tier	Role	MSP Convention	Ports	Purpose
1	Issuer	`issuer<N>MSP`	7051 (peer), 7054 (CA)	Registry operator, device management, audit

Tier	Role	MSP Convention	Ports	Purpose
2	Producer	`producer<N>MSP`	9051 (peer), 8054 (CA)	Energy production, GO creation, conversion
3	Consumer	`consumer<N>MSP`	11051 (peer), 9054 (CA)	GO receipt, cancellation, certificate verification

New Network Files

File	Purpose
`network/configtx.yaml`	Channel configuration with 3 org types, Raft consensus, anchor peers
`network/base.yaml`	Base Docker service definitions (peer-base, orderer-base, couchdb-base)
`network/docker/docker-compose-orderer.yaml`	4-node Raft orderer cluster
`network/docker/docker-compose-issuer.yaml`	Issuer organization peer + CouchDB
`network/docker/docker-compose-producer.yaml`	Producer org template (reusable for N producers)
`network/docker/docker-compose-consumer.yaml`	Consumer org template (reusable for N consumers)
`network/docker/docker-compose-ca.yaml`	4 Fabric CAs (issuer, producer, consumer, orderer)
`network/scripts/network-up.sh`	Bring up the entire network
`network/scripts/network-down.sh`	Tear down all containers, volumes, crypto
`network/scripts/deploy-chaincode.sh`	Full lifecycle chaincode deployment (package → install → approve → commit)

Private Data Collections

Collection	Member Policy	Cross-Access
`publicGOcollection`	All 3 org types	—
`privateDetails-issuer1MSP`	issuer1MSP only	—
`privateDetails-producer1MSP`	producer1MSP + issuer1MSP	Issuer can audit
`privateDetails-consumer1MSP`	consumer1MSP + issuer1MSP	Issuer can audit

🖥 Application — Backend (Express.js + TypeScript)

New Files

File	Purpose
`src/index.ts`	Express server entry point, route registration, CORS, health check
`src/fabric/gateway.ts`	Fabric Gateway connection via `@hyperledger/fabric-gateway` + gRPC
`src/fabric/contracts.ts`	Contract accessor helpers (one per named contract)
`src/middleware/auth.ts`	JWT authentication + role-based middleware
`src/middleware/error.ts`	Global error handler
`src/middleware/logger.ts`	Winston logger
`src/types/index.ts`	TypeScript interfaces mirroring Go chaincode structs
`src/routes/auth.ts`	`POST /api/auth/login` — org+user login, JWT issuance
`src/routes/devices.ts`	CRUD for devices (register, list, get, revoke, suspend, reactivate)
`src/routes/guarantees.ts`	Create electricity/hydrogen GOs, list, read public/private
`src/routes/transfers.ts`	Transfer single GO or by amount (electricity + hydrogen)
`src/routes/conversions.ts`	Hydrogen backlog management, issue hGO from backlog
`src/routes/cancellations.ts`	Cancel GOs, list statements, verify certificates
`src/routes/queries.ts`	Direct query passthrough to chaincode

File	Purpose
`.env.example`	Environment variable template

REST API Summary (25 endpoints)

```
POST   /api/auth/login                      → JWT token
POST   /api/devices                         → Register device (issuer)
GET    /api/devices                          → List devices
GET    /api/devices/:id                      → Get device
PUT    /api/devices/:id/revoke               → Revoke device (issuer)
PUT    /api/devices/:id/suspend              → Suspend device (issuer)
PUT    /api/devices/:id/reactivate           → Reactivate device (issuer)
POST   /api/guarantees/electricity           → Create electricity GO (producer)
POST   /api/guarantees/hydrogen              → Create hydrogen GO (producer)
GET    /api/guarantees                       → List all GOs
GET    /api/guarantees/:id                   → Read public GO data
GET    /api/guarantees/:id/private           → Read private GO details
POST   /api/transfers                        → Transfer single GO
POST   /api/transfers/electricity-by-amount → Transfer eGO by MWh
POST   /api/transfers/hydrogen-by-amount    → Transfer hGO by kg
POST   /api/conversions/backlog              → Add to hydrogen backlog (producer)
POST   /api/conversions/issue                → Issue hGO from backlog (producer)
GET    /api/conversions/backlog              → Query backlog (producer)
POST   /api/cancellations/electricity        → Cancel electricity GO
POST   /api/cancellations/hydrogen           → Cancel hydrogen GO
GET    /api/cancellations                    → List cancellation statements
POST   /api/cancellations/verify             → Verify certificate
GET    /api/queries/ego-list                 → All electricity GOs
GET    /api/queries/hgo-list                 → All hydrogen GOs
GET    /api/health                           → Health check
```

🎨 Application — Frontend (React 18 + Vite + Tailwind CSS)

New Files

File	Purpose
`src/main.tsx`	React app entry, router + auth provider
`src/App.tsx`	Route definitions, protected routes
`src/api.ts`	Axios client with JWT interceptor
`src/types.ts`	Frontend type definitions
`src/context/AuthContext.tsx`	Auth state management (login, logout, token)
`src/components/Layout.tsx`	Sidebar nav with role-aware menu items
`src/pages/LoginPage.tsx`	Org + user login form
`src/pages/DashboardPage.tsx`	Stats cards (GO counts, devices), quick actions
`src/pages/DevicesPage.tsx`	Device table + registration form (issuer)
`src/pages/GuaranteesPage.tsx`	Tabbed eGO/hGO list + create form (producer)
`src/pages/TransfersPage.tsx`	Transfer modes: single, eGO-by-amount, hGO-by-amount
`src/pages/ConversionsPage.tsx`	Add backlog + issue hydrogen GO (producer)
`src/pages/CertificatesPage.tsx`	Cancel GOs + verify certificates

Role-Based Navigation

Page	Issuer	Producer	Consumer
Dashboard	✓	✓	✓

Page	Issuer	Producer	Consumer
Devices	✓	✓	—
Guarantees	✓	✓	✓
Transfers	—	✓	✓
Conversions	—	✓	—
Certificates	✓	✓	✓

🗀 File Inventory

New Files (70 total)

Category	Count	Files
Architecture doc	1	`ARCHITECTURE.md`
Chaincode (Go)	18	`main.go`, `go.mod`, 5× assets, 3× access, 3× util, 6× contracts
Collections	1	`collection-config.json`
Network config	10	`configtx.yaml`, `base.yaml`, 5× Docker Compose, 3× scripts
Backend (TS)	14	`package.json`, `tsconfig.json`, `.env.example`, 3× fabric, 3× middleware, 1× types, 7× routes, `index.ts`
Frontend (TSX)	14	`package.json`, `tsconfig.json`, `tsconfig.node.json`, `vite.config.ts`, `tailwind.config.js`, `postcss.config.js`, `index.html`, `index.css`, `vite-env.d.ts`, `main.tsx`, `App.tsx`, `api.ts`, `types.ts`, `AuthContext.tsx`, `Layout.tsx`, 7× pages
Project docs	2	`architecture-diagrams.md`, `CHANGELOG-v2.1.md`

Preserved Files

The original `version1/` directory remains fully intact as a reference, including all DOCS.md documentation files.

🔧 Dependencies

Chaincode (Go 1.22.1)

Module	Version
`fabric-chaincode-go`	v0.0.0-20240605
`fabric-contract-api-go`	v1.2.2
`fabric-protos-go`	v0.3.3

Backend (Node.js)

Package	Version
`@hyperledger/fabric-gateway`	^1.5.0
`@grpc/grpc-js`	^1.10.0
`express`	^4.19.2
`jsonwebtoken`	^9.0.2
`winston`	^3.13.0
`typescript`	^5.4.0

Frontend

Package	Version
`react`	^18.3.1
`react-router-dom`	^6.23.0
`axios`	^1.7.0
`vite`	^5.2.0
`tailwindcss`	^3.4.3
`lucide-react`	^0.378.0

➡ SOON Roadmap (Planned)

- ☐ `go.sum` generation (requires Go toolchain)
- ☐ Chaincode unit tests (per contract)
- ☐ Fabric CA enrollment integration in backend
- ☐ End-to-end integration tests
- ☐ CI/CD pipeline (GitHub Actions)
- ☐ Caliper benchmark update for v2 chaincode
- ☐ Dynamic org onboarding script (`add-org.sh`)
- ☐ Production Docker Compose with resource limits
- ☐ Audit dashboard (issuer-only page)

Committed as `3d1a407` on 2026-03-21 — 70 files, 6,652 insertions.